

# Chain Rule

Connected functions ke through gradient ko pass karne ka tarika

## 01 Opening Hook

### STORY TIME

Socho ek factory assembly line hai – raw material se part A banta hai, part A se part B, aur part B se final product. Agar tumhe jaanna hai 'raw material mein chhota sa change final product ko kitna affect karega', tumhe har step ka effect multiply karna padega. Yehi hai **Chain Rule**.

## 02 Why Does This Concept Exist?

### THE PROBLEM

Neural networks mein data multiple layers se guzarta hai (function ke andar function). Agar humein pehli layer ke weight ka final loss par effect jaanna hai, humein saare intermediate layers ke through 'effect' ko chain karna padta hai. Chain rule invent hua taaki hum nested functions ke derivatives systematically calculate kar sakein.

## 03 Explain Like I'm 10

### SIMPLE WORDS

Socho tumhari height tumhare age par depend karti hai, aur tumhara shoe size tumhari height par depend karta hai. Agar tumhe jaanna hai 'age badhne se shoe size kitna badhega', tumhe dono relationships (age  $\rightarrow$  height, height  $\rightarrow$  shoe size) ko multiply karna padega.

Neural network mein bhi loss weight par indirectly depend karta hai (weight  $\rightarrow$  layer1 output  $\rightarrow$  layer2 output  $\rightarrow$  ...  $\rightarrow$  loss), aur chain rule saare in steps ko connect karta hai.

## 04 Build Intuition First (No Maths Yet)

### FEEL IT FIRST

Chain rule kehta hai: agar  $y$ ,  $u$  par depend karta hai, aur  $u$ ,  $x$  par depend karta hai, toh  $dy/dx = dy/du * du/dx$ . Neural networks mein yeh operation baar baar (layer by layer) apply hoti hai backward direction mein – isi ko backpropagation kehte hain.

## 05 Ab Thoda Maths

### FORMULA TIME

Formula:

Agar  $y = f(g(x))$ , toh  $dy/dx = f'(g(x)) * g'(x)$  – 'outer function ka derivative' times 'inner function ka derivative'.

Multi-layer:

Neural network mein:  $dLoss/dW1 = dLoss/dLayer3 * dLayer3/dLayer2 * dLayer2/dLayer1 * dLayer1/dW1$  – saare intermediate derivatives ko multiply karte jaate hain.

## 06 Visualize Everything

### PICTURE IT

```
x --> [Layer1] --> [Layer2] --> [Layer3] --> Loss
```

Chain Rule (backward):

```
dLoss/dLayer1_weights =
```

```
    dLoss/dLayer3 * dLayer3/dLayer2 * dLayer2/dLayer1 * dLayer1/dweights  
(har arrow ka derivative multiply hota hai peeche ki taraf)
```

## 07 AI Connection

### WHERE AI USES THIS

Backpropagation: Poora backpropagation algorithm chain rule ka hi practical implementation hai – gradients ko output se input tak layer-by-layer 'chain' karke propagate kiya jaata hai.

Deep Networks: Jitni zyada layers (jitna 'deep' network), utni lambi chain rule ki chain – isliye deep networks mein vanishing/exploding gradient jaisi problems aati hain.

## 08 Real Industry Example

### REAL COMPANIES

All AI companies: Har deep learning model ka training chain rule (backpropagation) ke bina possible nahi hai – yeh literally poore field ka foundation hai.

## 09 Explain Like I'm Going To Build It

### HOW YOU'D CODE IT

PyTorch ka autograd chain rule ko automatically apply karta hai. Jab tum `loss.backward()` call karte ho, PyTorch computation graph ko reverse order mein traverse karta hai, har step par chain rule apply karke gradients accumulate karta hai.

## 10 Common Misconceptions

### BEGINNER MISTAKES

Galat soch: Chain rule sirf 2 functions ke liye kaam karta hai.

Sahi baat: Chain rule kitne bhi nested functions ke liye kaam karta hai – deep neural networks mein hazaron layers tak chain ho sakti hai.

Galat soch: Backpropagation aur chain rule alag concepts hain.

Sahi baat: Backpropagation basically chain rule ka hi systematic, efficient application hai neural networks ke context mein.

## 11 Interview Questions

### INTERVIEW CORNER

[Beginner] Q: Chain rule kya karta hai?

A: Nested functions (function ke andar function) ka derivative calculate karta hai, har layer ke derivative ko multiply karke.

[Intermediate] Q: Backpropagation aur chain rule mein kya relation hai?

A: Backpropagation neural network training mein chain rule ka practical, efficient implementation hai – yeh output se input tak gradients ko layer-by-layer 'chain' karta hai.

[Advanced] Q: Deep networks mein chain rule vanishing gradient problem kaise create karta hai?

A: Agar har layer ka derivative chhota (jaise  $<1$ ) hai, chain rule unhe multiply karta jaata hai jitni layers deep jaati hain, jisse final gradient exponentially chhota ho jaata hai – shuruwaati layers ka gradient itna chhota ho jaata hai ki learning practically ruk jaati hai.

## 12 Exercises

### PRACTICE TIME

[Easy]  $y = (2x+1)^2$ . Chain rule use karke  $dy/dx$  nikalo.

Solution: Outer:  $2*(2x+1)$ , Inner: 2. Total =  $2*(2x+1)*2 = 4(2x+1) = 8x+4$ .

[Medium] Agar Layer2 ka derivative 0.5 hai aur Layer1 ka derivative 0.3 hai, combined derivative kya hoga?

Solution:  $0.5 * 0.3 = 0.15$  (multiply hota hai chain rule mein).

[Hard] Agar ek 50-layer deep network mein har layer ka average derivative 0.9 hai, first layer tak gradient kitna 'shrink' hoga approximately?

Solution:  $0.9^{50} \approx 0.005$  – matlab gradient bahut chhota ho jaayega (vanishing gradient), jisse first layers bahut slowly seekhengi.

## 13 Golden Summary

| Concept    | Easy Meaning                                      |
|------------|---------------------------------------------------|
| Chain Rule | Nested functions ke derivatives ko multiply karna |
| Formula    | $dy/dx = dy/du * du/dx$                           |

|                    |                                                                                |
|--------------------|--------------------------------------------------------------------------------|
| Backpropagation    | Chain rule ka neural network mein practical implementation                     |
| Vanishing Gradient | Chhote derivatives multiply hoke bahut chhote ho jaate hain deep networks mein |
| Deep Networks      | Jitni zyada layers, utni lambi chain                                           |

## 14 One-Line Memory Trick

**“Chain rule ek zanjeer (chain) hai jo har link ka effect aage tak carry karti hai.”**